

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a404a145942d37ae5.jsonl`  
- SHA-256: `eb59f11cba390640499c8068338530f058ac1f8775daa93c98f381639e3cbb1e`  
- Source modified: `2026-06-10T06:08:31+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:06:42.243Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("testing") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"FastAPI serving layer (/api/process, /api/query, /api/compile) has zero in-process test coverage","severity":"critical","area":"API / backend.main","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py (endpoints at lines 87-207)","evidence":"No test imports backend.main and there is no TestClient usage anywhere in tests/ (grep for 'TestClient|backend.main' in tests/ matches only docstrings in test_import_smoke.py). backend/main.py (426 lines) does not appear at all in the pytest-cov report (--cov=backend, 68% total). The only guard is tests/test_import_smoke.py:33-55, which imports backend.main in a subprocess to catch import errors ? it executes no endpoint logic. Untested logic includes: validation-failure short-circuit, segmenter selection with 'motion' fallback (line ~104 getattr default), Failure-result handling, and the finally-block report emission that mutates the response dict (lines 160-172).","why_it_matters":"This is the orchestration glue where most integration regressions surface (wrong status strings, broken response shapes, report paths not attached, segmenter fallback drift). The next committed work item is the async job model, which rewrites exactly this file ? there is currently no test harness to detect behavior changes during that refactor. A FastAPI TestClient suite with mocked segmenter/db would be cheap and offline."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:06:48.579Z)

```
1      import os
2      import sys
3      import argparse
4      import json
5      import logging
6      import uvicorn
7      from dotenv import load_dotenv
8
9      load_dotenv()
10
11     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
12     logging.basicConfig(
13         level=logging.INFO,
14         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
15         datefmt='%H:%M:%S'
16     )
17     logger = logging.getLogger(__name__)
18     from fastapi import FastAPI, HTTPException
19     from fastapi.middleware.cors import CORSMiddleware
20     from pydantic import BaseModel
21     from typing import List, Dict, Any, Optional
22
23     from fastapi.staticfiles import StaticFiles
24     from fastapi.responses import HTMLResponse
25     from backend.pipeline.validators.base import registry
26     from backend.infrastructure.database import Database
27     from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
28     from backend.pipeline.stitcher.compiler import VideoCompiler
29     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30     from backend.config import load_config as load_app_config, AppConfig # new typed config
31     from backend.results import Result, Failure # standardized error/result contract
32     from backend.reporting import emit_indexer_report
33
34     app = FastAPI(title="Sports Highlight Indexer API")
35
36     # Enable CORS for local web interface
37     app.add_middleware(
38         CORSMiddleware,
39         allow_origins=["*"],
40         allow_credentials=True,
41         allow_methods=["*"],
42         allow_headers=["*"],
43     )
44
45     # Serves static frontend files directly (now under api/ per approved structure)
46     os.makedirs("backend/api/static", exist_ok=True)
47     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
48
49     @app.get("/", response_class=HTMLResponse)
50     def serve_index():
51         index_path = "backend/api/static/index.html"
52         if os.path.exists(index_path):
53             with open(index_path, "r", encoding="utf-8") as f:
54                 return f.read()
55         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
56
57     # Global variables initialized at startup
58     db_instance: Optional[Database] = None
59     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
60
61     # Legacy thin wrapper for any remaining direct calls during transition.
```

```

62     # New code should import load_app_config / AppConfig from backend.config directly.
63     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
64         cfg = load_app_config(config_path)
65         return cfg.model_dump(mode="json")
66
67     # --- API Models ---
68     class ProcessRequest(BaseModel):
69         video_path: str
70         player_pool: Optional[List[str]] = None
71         max_frames: Optional[int] = None
72         segmenter: Optional[str] = None
73
74     class QueryRequest(BaseModel):
75         video_id: str
76         server: Optional[str] = None
77         receiver: Optional[str] = None
78         duration_min: Optional[float] = None
79         event_type: Optional[str] = None
80
81     class CompileRequest(BaseModel):
82         video_id: str
83         segment_ids: List[int]
84         output_filename: str
85
86     # --- API Endpoints ---
87     @app.get("/api/config")
88     def get_config():
89         return global_config
90
91     @app.post("/api/process")
92     def process_video(req: ProcessRequest):
93         if not os.path.exists(req.video_path):
94             raise HTTPException(status_code=404, detail=f"Video file not found at
95 '{req.video_path}'")
96
97         video_id = compute_video_id(req.video_path)
98         was_reingest = db_instance.get_video(video_id) is not None
99
100        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
101        report)
102        logger.info(f"Running validations for {req.video_path}...")
103        val_results, val_context = registry.run_all_with_context(req.video_path,
104        global_config)
105        failures = [r for r in val_results if not r.passed]
106
107        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
108        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
109        "motion")
110        seg_name = req.segmenter or default_seg
111        segmenter_actual = None
112        segmentation_ran = False
113        response: Optional[Dict[str, Any]] = None
114        try:
115            if failures:
116                db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
117        val_results])
118                response = {
119                    "video_id": video_id,
120                    "status": "failed",
121                    "message": "Video failed validation checks.",
122                    "results": [r.dict() for r in val_results],
123                }
124            return response
125
126        # 2. Run segmenter & indexer

```

```

122         print(f"[API] Video passed checks. Segmenting and indexing...")
123         db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
124
125         segmenter_cls = segmenter_registry.get(seg_name)
126         if not segmenter_cls:
127             available = list(segmenter_registry.list_available().keys())
128             raise HTTPException(
129                 status_code=400,
130                 detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
131             )
132
133         print(f"[API] Using segmenter: {seg_name}")
134         segmenter_actual = seg_name
135         segmenter = segmenter_cls(db_instance, global_config)
136         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
137         segmentation_ran = True
138
139         if isinstance(result, Failure):
140             print(f"[API] Segmenter failed: {result.message}")
141             response = {
142                 "video_id": video_id,
143                 "status": "failed",
144                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
145                 "results": [r.dict() for r in val_results],
146                 "play_segments": [],
147             }
148             return response
149
150         segments = result.value if hasattr(result, "value") else result
151
152         response = {
153             "video_id": video_id,
154             "status": "success",
155             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
156             "results": [r.dict() for r in val_results],
157             "play_segments": segments,
158         }
159         return response
160     finally:
161         # Always emit the per-video observability report (next to the video, or to
162         # report.output_dir). Never raises. Surface the paths in the response.
163         paths = emit_indexer_report(
164             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
165             val_results=val_results, val_context=val_context,
166             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
167             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
168         )
169         if paths and isinstance(response, dict):
170             response["reports"] = paths
171
172     @app.post("/api/query")
173     def query_play_segments(req: QueryRequest):
174         filters = {
175             "server": req.server,
176             "receiver": req.receiver,
177             "duration_min": req.duration_min,
178             "event_type": req.event_type
179         }
180         segments = db_instance.query_play_segments(req.video_id, filters)
181         return {"play_segments": segments}

```

```

182
183 @app.post("/api/compile")
184 def compile_highlights(req: CompileRequest):
185     video = db_instance.get_video(req.video_id)
186     if not video:
187         raise HTTPException(status_code=404, detail="Indexed video record not found.")
188
189     # Retrieve matching play segments from db
190     all_segments = db_instance.query_play_segments(req.video_id, {})
191     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
192
193     if not matched_segments:
194         raise HTTPException(status_code=400, detail="No matching play segments found to
195 stitch.")
196
197     # Extract start/end time pairs
198     time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]
199
200     # Run compiler
201     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
202     or "output"
203     compiler = VideoCompiler(output_dir)
204     try:
205         out_path = compiler.compile_highlights(video["filepath"], time_pairs,
206 req.output_filename)
207         return {"status": "success", "filepath": out_path}
208     except Exception as e:
209         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
210 {str(e)}")
211
212 # --- CLI Debug Utility & Orchestration ---
213 def run_cli_diagnose_clip(video_path: str, timestamp: float):
214     """CLI debugging utility to examine segment properties around a timestamp."""
215     print("=" * 60)
216     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
217     print("=" * 60)
218
219     cfg = load_config() # legacy wrapper still works, returns dict for now
220
221     # 1. Validation check diagnostics
222     print("[DIAGNOSTIC] Running validation framework...")
223     results = registry.run_all(video_path, cfg)
224     for r in results:
225         status_symbol = "[PASS]" if r.passed else "[FAIL]"
226         print(f" {status_symbol} {r.validator_name}: {r.message}")
227         if r.details:
228             print(f"      Details: {r.details}")
229
230     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
231     print("-" * 60)
232     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
233     import cv2
234     cap = cv2.VideoCapture(video_path)
235     if not cap.isOpened():
236         print("[FAIL] OpenCV could not open video.")
237         return
238
239     fps = cap.get(cv2.CAP_PROP_FPS)
240     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
241
242     start_frame = max(0, int((timestamp - 3.0) * fps))
243     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
244
245     # Measure frame-by-frame changes in sample region
246     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)

```

```

243     prev_gray = None
244     motions = []
245
246     for f in range(start_frame, end_frame):
247         ret, frame = cap.read()
248         if not ret:
249             break
250         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
251         gray = cv2.GaussianBlur(gray, (21, 21), 0)
252
253         if prev_gray is not None:
254             diff = cv2.absdiff(prev_gray, gray)
255             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
256             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
257             motions.append(motion_ratio)
258             prev_gray = gray
259
260     cap.release()
261
262     if motions:
263         avg_motion = sum(motions) / len(motions)
264         # Support both legacy dict (from wrapper) and future direct model
265         if hasattr(cfg, "indexing"):
266             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
267         else:
268             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
269         print(f" Sample Frame Count: {len(motions)}")
270         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
271         if avg_motion > threshold:
272             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
273         else:
274             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
275     else:
276         print(" [ERROR] No visual frames were extracted in the sample range.")
277
278     print("=" * 60)
279
280     def main():
281         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
282         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
283         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
284         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
285         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
286         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
287         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
288         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
289
290         available_segmenters = list(segmenter_registry.list_available().keys())
291         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
292         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
293         parser.add_argument("--trim-end", type=float, help="End time in seconds for the

```

```

debug trim segment feature")
294     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
295
296     args = parser.parse_args()
297
298     if args.setup:
299         # Invoke setup diagnostics
300         import subprocess
301         print("[INFO] Invoking setup.py...")
302         subprocess.run([sys.executable, "setup.py"])
303         return
304
305     if args.trim_start is not None and args.trim_end is not None:
306         if not args.video_path:
307             print("[ERROR] Please provide --video-path to extract a segment.")
308             return
309
310         # Determine output path
311         out_filename = args.trim_output or "trimmed_segment.mp4"
312         if os.path.isabs(out_filename):
313             out_path = out_filename
314             out_dir = os.path.dirname(out_path)
315             out_name = os.path.basename(out_path)
316         else:
317             out_dir = args.output_dir
318             out_path = os.path.join(out_dir, out_filename)
319             out_name = out_filename
320
321         os.makedirs(out_dir, exist_ok=True)
322         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
323
324         compiler = VideoCompiler(out_dir)
325         try:
326             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
327             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
328         except Exception as e:
329             print(f"[ERROR] Failed to compile trimmed segment: {e}")
330         return
331
332     if args.debug_clip is not None:
333         if not args.video_path:
334             print("[ERROR] Please provide --video-path to debug a specific clip.")
335             return
336         run_cli_diagnose_clip(args.video_path, args.debug_clip)
337         return
338
339     # Initialize Global Config and DB
340     global global_config, db_instance
341     global_config = load_app_config() # returns typed AppConfig
342
343     # The CLI --output-dir overrides the configured output directory. It now lives
344     # on the typed model (OutputConfig.output_dir), so every consumer ? including
345     # /api/compile ? reads the same value instead of a write-only runtime hack.
346     global_config.output.output_dir = args.output_dir
347     os.makedirs(global_config.output.output_dir, exist_ok=True)
348
349     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
350     db_instance = Database(db_path)
351
352     # CLI processing mode (no web UI needed)
353     if args.video_path and not args.server:

```

```

354         print(f"[CLI] Processing video file: {args.video_path}")
355         if not os.path.exists(args.video_path):
356             print(f"[FAIL] Video file not found: {args.video_path}")
357             return
358
359         video_id = compute_video_id(args.video_path)
360         was_reingest = db_instance.get_video(video_id) is not None
361
362         # Validate (with-context variant surfaces ffprobe_meta for the report)
363         print("[CLI] Validating...")
364         val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
365         failures = [r for r in val_results if not r.passed]
366
367         # Save video status
368         status = "failed" if failures else "processed"
369         db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])
370
371         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
372         segmenter_actual = None
373         segmentation_ran = False
374         try:
375             print("\n" + "="*40)
376             print("VALIDATION SUMMARY")
377             print("="*40)
378             for r in val_results:
379                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
380                 print(f"{status_symbol} {r.validator_name}: {r.message}")
381             print("="*40 + "\n")
382
383             if failures:
384                 print("[FAIL] Video failed checks. Indexing halted.")
385                 return
386
387             # Index
388             segmenter_cls = segmenter_registry.get(seg_name)
389             if not segmenter_cls:
390                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
391                 return
392
393             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
394             segmenter_actual = seg_name
395             segmenter = segmenter_cls(db_instance, global_config)
396             result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
397             segmentation_ran = True
398
399             if isinstance(result, Failure):
400                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
401                 print(f"[FAIL] Indexing halted for video.")
402                 segments = []
403             else:
404                 segments = result.value if hasattr(result, "value") else result
405                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
406             finally:
407                 # Always emit the per-video observability report. Never raises.
408                 paths = emit_indexer_report(
409                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
410                     val_results=val_results, val_context=val_context,
411                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
412                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,

```



```

413         )
414     if paths:
415         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
416         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
417         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
418     return
419
420     # Start Local Server Mode
421     if args.server or not args.video_path:
422         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
423         uvicorn.run(app, host="127.0.0.1", port=args.port)
424
425     if __name__ == "__main__":
426         main()
427

```

3. user (2026-06-10T06:06:50.454Z)

```

README.md:64:python -m backend.main --server --port 8000
README.md:72:python -m backend.main --video-path "C:/path/to/my_video.mp4"
README.md:78:python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter
yolo_hybrid
README.md:79:python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
README.md:80:python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
README.md:86:python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
README.md:93:python -m backend.main --video-path "C:/path/to/my_video.mp4" \
README.md:183:    python -u -m backend.main --video-path "C:/path/to/video.mp4"
README.md:193:python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4"
--max-frames 300
README.md:207:    python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter
yolo_hybrid
README.md:210:    python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini
README.md:213:    python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
TEST_RUN_SUMMARY.md:60:    python -u -m backend.main --video-path
"C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-frames 500
docs\DEFERRED_HARDENING_PLAN.md:36:- Console entry points: `indexer = backend.main:main` (so
`indexer --server` works),
tests\test_import_smoke.py:6:backend.main pulls in) used to go unnoticed.
tests\test_import_smoke.py:9:anything, and the backend.main import runs in a subprocess with any
missing
tests\test_import_smoke.py:34:    """backend.main (FastAPI app, /api/compile) must be
importable.
tests\test_import_smoke.py:45:        "import backend.main\n"
tests\test_import_smoke.py:54:        "import backend.main failed:\n" + result.stderr.strip()

```

4. user (2026-06-10T06:06:59.872Z)

```

1    """Import/syntax smoke tests for the backend package.
2
3    Guards against a SyntaxError (or module-level import error) shipping while the
4    suite stays green: most of the suite imports individual backend modules, so a
5    broken file that nothing imports (e.g. the stitcher compiler, which only
6    backend.main pulls in) used to go unnoticed.
7
8    Both tests are offline and dependency-light: the syntax sweep never imports
9    anything, and the backend.main import runs in a subprocess with any missing
10   heavy deps (cv2/torch/numPy) stubbed, so they also pass in environments
11   without the GPU stack.
12   """
13
14   import py_compile
15   import subprocess
16   import sys
17   from pathlib import Path

```

```

18
19 REPO_ROOT = Path(__file__).resolve().parents[1]
20
21
22 def test_backend_package_has_no_syntax_errors():
23     """Every .py file under backend/ must byte-compile."""
24     failures = []
25     for path in sorted((REPO_ROOT / "backend").rglob("*.py")):
26         try:
27             py_compile.compile(str(path), doraise=True)
28         except py_compile.PyCompileError as exc:
29             failures.append(f"{path.relative_to(REPO_ROOT)}: {exc.msg}")
30     assert not failures, "Syntax errors in backend/:\n" + "\n".join(failures)
31
32
33 def test_backend_main_imports():
34     """backend.main (FastAPI app, /api/compile) must be importable.
35
36     Runs in a subprocess so stubbing missing heavy deps cannot leak into this
37     pytest process; cwd=repo root puts the package on sys.path.
38     """
39     prelude = (
40         "import importlib.util, sys\n"
41         "from unittest import mock\n"
42         "for name in ('cv2', 'torch', 'numpy'):\n"
43         "    if importlib.util.find_spec(name) is None:\n"
44         "        sys.modules[name] = mock.MagicMock()\n"
45         "import backend.main\n"
46     )
47     result = subprocess.run(
48         [sys.executable, "-c", prelude],
49         cwd=str(REPO_ROOT),
50         capture_output=True,
51         text=True,
52     )
53     assert result.returncode == 0, (
54         "import backend.main failed:\n" + result.stderr.strip()
55     )
56

```

5. user (2026-06-10T06:07:01.782Z)

93: `POST /api/process` runs validate ? segment ? report **inline**, blocking the
97: - `POST /api/process` returns **202 + `job\_id`**; add `GET /api/jobs/{job\_id}`

6. user (2026-06-10T06:07:11.359Z)

docs\GOLDEN_SET_PHASE1_VIDEOS.md:12:[Omitted long matching line]
docs\GOLDEN_SET_PHASE1_VIDEOS.md:30:| Tilt | **~30° down** | Optimal coverage, low keystone |
docs\CODE_MAP.md:169:- `@backend/annotations/automated\_provider.py::AutomatedProvider`
(`auto`; GS-4 stub; injectable `::Labeler`, in-process `\_jobs` store, `::warn\_if\_same\_lineage`
ORACLE RULE ? only warns, does not block; default `\_not\_configured` RAISES NotImplementedError).
? get via `annotation\_registry.get('auto', labeler=fn, lineage='...')`; registered as `auto`
only because no-arg ctor succeeds.
docs\CODE_MAP.md:177:[Omitted long matching line]
docs\CODE_MAP.md:315:| `test\_gemini\_refine.py` | `eval.gemini\_refine` merge_overlaps /
_offset_segments (pure helpers, no API) |
docs\CODE_MAP.md:329:| ? stale `.pyc` only | `tests/\_\_pycache\_\_/test\_gemini\_api.\*` ? no live
source exists |
docs\DEFERRED_HARDENING_PLAN.md:31: - `[project.optional-dependencies] dev` ? `pytest`,
coverage tooling.
docs\DEFERRED_HARDENING_PLAN.md:112:- In-process store is not horizontally scalable ? acceptable
for the current
docs\ROADMAP.md:67:stage at **recall ~0.05 @ IoU 0.5** on `shuttle8` (median best coverage
of a

docs\PMF_MARKET_RESEARCH.md:42:players & clubs** (capital-rich, fastest-growing, but SwingVision already strong);

docs\PMF_MARKET_RESEARCH.md:169:Fastest-growing sport (19.8M US, +45.8% YoY, 2024); capital-rich; ~70,000 courts;

docs\PLATFORM_ARCHITECTURE.md:53:| Execution | pipeline runs ****in-process, synchronously**** in ``/api/process`` | blocks; can't dispatch to a user's VM/GPU; no job status |

docs\PLATFORM_ARCHITECTURE.md:138:- ****`local`**** ? in-process / local subprocess (today's behavior; dev + self-host).

docs\archives\COMMERCIAL_READINESS_REVIEW.md:50:All delivered as separate, small PRs with the required discipline (full test suite via miniconda + ``run_all_tests.py --cov``, coverage maintained at ~64%, docs updated in-plan + CODE_MAP, easy to carry forward).

docs\archives\research\AUDIO_E1_FINDINGS.md:20:| Rally coverage (rallies with ?2 detected hits) | ****100%**** (36/36) | ****100%**** (7/7) |

docs\archives\research\AUDIO_E1_FINDINGS.md:28:| ?at k=2.5 / best-coverage k=4 | 1.3× / 2.8× (coverage drops to 30/36) | 1.5× |

docs\archives\research\AUDIO_E1_FINDINGS.md:58:4. ****Separate the recall role from the precision role.**** Audio was excellent at recall (100% coverage) and poor

docs\archives\decisions\DECISIONS.md:205:- ****72%**** of rallies have *some* candidate overlap, yet the ****median best coverage of a**

docs\archives\decisions\DECISIONS.md:265:[Omitted long matching line]

docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md:108:- ``k`` (threshold) is the only real lever and trades coverage for discrimination (~3× costs half the rallies).

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:177:Low-regret items 3?6 + the three scaffolding maturation items (Detector finish #49, Reports-based Input Quality #50, minimal Value Layer #51) all completed with per-item branches/PRs, full miniconda test + coverage runs (maintained ~64%), and docs updates.

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:191:All three recommended scaffolding items completed as separate PRs with full tests/coverage/docs discipline. Scaffolding now matured for iteration.

docs\archives\checkpoints\2026-06-logging-reporting\README.md:19:- Future quality iterations (new detectors, audio fusion, model changes) must not regress the "never raises", footgun coverage, or report quality.

docs\archives\checkpoints\2026-06-eval-baseline\README.md:21:- Coverage of new data in the golden set vs. eval splits.

7. user (2026-06-10T06:07:12.020Z)

```

80     - Small corpus ? tiny held-out set ? high metric variance; the guardrail is correct
81       but the numbers will be noisy until the golden set grows.
82     - MD5-only dedup is insufficient (re-encodes evade it) ? must track match identity.
83
84     ### Verification
85     - Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
86     - Integration: a deliberately-leaked manifest fails the training endpoint.
87
88     ---
89
90     ## 3. Async job model (debt #16)
91
92     ### Problem
93     `POST /api/process` runs validate ? segment ? report **inline**, blocking the
94     request for the full duration of a long video. Unworkable as a hosted/product API.
95
96     ### Approach (minimal first slice ? single self-hosted box)
97     - `POST /api/process` returns **202 + `job_id`**; add `GET /api/jobs/{job_id}`
98       returning {status, progress, result, reports}`.
99     - Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
100      (segmenters already use threads internally for AI handoff, so this fits); leave a
101      real queue (RQ/Celery) for later if multi-process is needed.
102     - Persist job state in the existing SQLite DB via the migration ladder
103      (`backend/infrastructure/database.py::init_db`, bump `PRAGMA user_version` 1 ? 2,
104      add a `jobs` table). The observability report becomes the job's artifact.
105     - Update the static UI to poll `GET /api/jobs/{id}` instead of blocking.
106
107     ### Risks

```

```

108 - **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
109   `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
110   confirm it's safe under the executor.
111 - Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
112 - In-process store is not horizontally scalable ? acceptable for the current
113   single-box self-hosted target; call it out so it isn't mistaken for production-HA.
114
115   ### Verification
116   - Submit a job ? poll ? DONE with the same segments the sync path produced.
117   - Server restart mid-job leaves a recoverable/clearly-FAILED job, not a hang.
118
119   ---
120
121   ## Out of this doc
122   - **Detector/WSL abstraction (#6):** in flight (PR #49).
123   - **Unify hybrid segmenters (#7 in LOW_REGRET):** owner-deferred.
124   - **ByteTrack `supervision<0.30.0` migration + real-footage validation (#14/#15):**
125     need a GPU/WSL box and real video ? owner-machine work, tracked separately.
126

```

8. user (2026-06-10T06:07:22.474Z)

Found 7 files
tests\test_reporting.py
tests\test_base.py
tests\test_yolo_hybrid.py
tests\test_reporting_harvest.py
tests\test_tracknet_hybrid.py
tests\test_gemini.py
tests\test_database.py

9. user (2026-06-10T06:07:23.205Z)

```

300 | `test_tracknet_runner.py` | `pipeline/detectors/tracknet_runner` (`to_wsl_mnt_path`,
`from_indexing_cfg`, parse + `trajectory_to_action_windows` shared brain, healthcheck,
run_predict, stage) |
301 | `test_rally_gate.py` | `pipeline/detectors/rally_gate` (axis/net estimate, crossing
count, gate kills single-toss) |
302 | `test_wasb_cache.py` | `pipeline/detectors/wasb_infer` resumable cache (serialize,
contiguity heal, manifest invalidation, atomic writes, `default_cache_dir`) |
303 | `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
304 | `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate,
`write_scaffold` |
305 | `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict;
also `@run_eval.py::main`/`get_file_md5` (exit codes, `--scaffold`/`--json`) |
306 | `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate
(micro/macro, zero-div) |
307 | `test_calibration.py` | `eval.calibration`
pct_improvement/kfold/select_best/cross_validate/LOVO/improvement_report |
308 | `test_calibrate_wasb.py` | `eval.calibrate_wasb`
load_golden_gts/make_predict_fn/grid/run, BASELINE |
309 | `test_calibrate_local.py` | `eval.calibrate_local` local grid + crossing gate, leave-
one-video-out |
310 | `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment/comparison CSV
schemas, mmss; export-> slicer reuse |
311 | `test_eval_training.py` | `eval.training` label_candidates / feature extraction /
build_training_set |
312 | `test_eval_classifier.py` | `eval.classifier.LogisticRegression`
fit/predict_proba/balanced/JSON round-trip |
313 | `test_eval_regression.py` | `eval.regression` baseline store/compare/gate; also
`@run_regression.py::main` (exit-code CI gate) |
314 | `test_distill_local.py` | `eval.distill_local` build_candidates / predict_rallies /
learned-vs-baseline |
315 | `test_gemini_refine.py` | `eval.gemini_refine` merge_overlaps / _offset_segments (pure
helpers, no API) |

```

```

316 | `test_hit_detector.py` | `pipeline/audio/hit_detector` (detect_hits, onset_envelope,
    | _moving_stats) + `eval/audio/e1_probe.cluster_hits` |
317 | `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label
    | parse |
318 | `test_video.py` | `utils.video` duration/chunk (subprocess mocked) |
319 | `test_geometry.py` | `utils.geometry` center/distance/velocity/normalised-velocity/IoU
    | |
320 | `test_database.py` | `infrastructure.database.Database`
    | schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |
321 | `test_config.py` | `config` (load_config defaults, save_default_config, AppConfig
    | defaults pinned, `extra` tolerance, `.get()` dict-compat) |
322 | `test_reporting_harvest.py` | `reporting` (`emit_indexer_report`,
    | `harvest.record_run`); ? `importorskip("obsreport")`; all 7 footgun rules + plain-dict-OR-
    | AppConfig + `report.enabled=False` |
323 | `test_reporting_spec_faq.py` | `reporting/spec.yaml` <-> `README.md` cross-ref
    | integrity (every `faq_ref` resolves; actionable rules cite a faq_ref); ?
    | `importorskip("obsreport")` |
324 | `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
    | (register/run_all/run_all_with_context` -> `(results,context)`; exception captured not raised)
    | |
325 | `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile (prompt contains
    | "decimal seconds"/chunk value; relevance config keys) |
326 | `test_providers.py` / `test_provider_gemini.py` | `providers` registry +
    | GeminiProvider lifecycle (`MAX_UPLOAD_MB` guard, non-list JSON -> [], cleanup delete) |
327 | `test_annotation_providers.py` / `test_automated_provider.py` | `annotations`
    | registry, FileProvider, AutomatedProvider (oracle/lineage); keystone one-scorer contract ->
    | `eval.metrics.score` |
328 | (no dedicated test) | individual `pipeline/validators/*` impls,
    | `pipeline/stitcher.compiler`, `main.py`, `run_process_and_stitch.py`,
    | `pipeline/detectors/wasb_runner`, `pipeline/segmenters/{wasb_hybrid,motion}`,
    | `eval/audio/e1_probe` (diagnostic) ? ? exercised only indirectly |
329 | ? stale `.pyc` only | `tests/__pycache__/test_gemini_api.*` ? no live source exists |
330

```

10. user (2026-06-10T06:07:55.721Z)

```

62: - `backend/api/` (FastAPI app, routes, static UI)
68:- ? **Deliverable 1 (Config)** ? `backend/config/` package (`models.py` + `loader.py`)
    | instead of a single `config.py`. Typed `AppConfig`, central loader with built-in defaults + file
    | override, `extra="allow"` for migration. Update `setup.py` + `main.py`. Add tests. Land this
    | before the layout move.
80:- ? **Item 3 (Error/Result contract) synergizes with the report.** Today the report INFERS
    | failures (e.g. `silent_provider_noop` = 0 segments + no API key). A structured Result/error from
    | providers/segmenters would let the report read the real reason as a first-class flag.
    | **Revisit:** with item 3.
84: #39/#40 are fixed: (1) `*/api/compile` now honors `--output-dir` and (2) `output_dir`
    | is a real field on

```

11. user (2026-06-10T06:08:25.526Z)

Structured output provided successfully

12. assistant (2026-06-10T06:08:30.907Z)

Verification complete ? structured output returned (isReal=false, high confidence: facts accurate but gap is documented in CODE_MAP.md:328 and "critical" severity is overstated).